

Betalningar → registrering i Visma Compact

Från rå Bankgiro-/Swish-fil till registrerade inbetalningar och månadsavstämning

Projekt: `C:\ekonomi\scripts` · Ett dokument som ersätter den tidigare tutorialen och översiktssidan · Sanningskälla: koden

Om detta dokument

Guiden är skriven utifrån vad koden *faktiskt* gör. Där äldre dokumentation sa något annat följer guiden koden och noterar avvikelser. Punkter märkta **OKLART** kräver ett beslut från dig (se ändringsloggen).

Innehåll

1. Översikt
2. Kom igång – förutsättningar, installation, konfiguration
3. Huvudflödet steg för steg
 1. Exportera rådata (Bankgiro & Swish)
 2. Rensa Bankgiro-filer
 3. Rensa Swish-filer
 4. Sammanställ Bankgiro
 5. Sammanställ Swish
 6. Skapa CSV (Bankgiro)
 7. Skapa CSV (Swish)
 8. Konvertera CSV till Visma-format
 9. Registrera inbetalningar i Visma
 10. Sammanställ & stäm av loggar
4. Alternativa & fristående flöden
5. Referens – alla skript, parametrar och flaggor
6. Felsökning
7. Ordlista
8. Ändringslogg & beslut som behövs

1. Översikt

Systemet tar emot inbetalningar från två källor – **Bankgiro** (insättningsuppgifter) och **Swish** (transaktionsexport från SEB) – rensar och sammanställer dem, extraherar fakturanummer och producerar ett gemensamt CSV-format. Det formatet konverteras till Visma-kolumner och registreras **halvautomatiskt som kundinbetalningar** i Visma Compact 6 (Kundreskontra → Inbetalningar), varefter körloggarna slås ihop till en månadsfil med dubblettkontroll och avstämning mot en export över betalda fakturor. Varje skript är ett eget steg som läser filer från en mapp och skriver nya filer utan att röra originalen.

Flödesdiagram



Två spår, samma format

Bankgiro och Swish har varsin rensnings- och sammanställningskedja men möts i exakt samma CSV-format. Därför kan resten av flödet (konvertering + registrering) hantera båda likadant.

Fristående / alternativa delar (se avsnitt 4): `payment_register.py` registrerar samma typ av betalningar i det webbaserade fakturasystemet i stället för i Visma, och `visma_import.py` registrerar

kundfakturor (inte inbetalningar) i Visma Kundbokning.

2. Kom igång

Förutsättningar

- **Windows** med **Python 3** (verifiera med `python --version`).
- **PowerShell** (kommandona nedan är skrivna för PowerShell).
- Projektet i `C:\ekonomi\scripts`.
- För registreringssteget: **Visma Compact 6** installerat och inloggat.
- För det alternativa webbfliödet: konto i webbfakturasystemet.

Installation

Öppna PowerShell i projektmappen och installera alla beroenden en gång:

```
cd C:\ekonomi\scripts
python -m pip install -r requirements.txt
```

(motsvarar `openpyxl pandas playwright pyautogui pywinauto pyperclip`).

Extra, beroende på vad du använder:

```
# Endast för det alternativa webbfliödet (payment_register.py):
python -m playwright install chromium

# Endast om du matar in gamla .xls-filer i Visma-registreringen:
python -m pip install xlrd
```

requirements.txt

Filen är uppdaterad och täcker nu hela flödet: `openpyxl`, `pandas`, `pyautogui`, `pywinauto`, `pyperclip` och `playwright` (samt `xlrd` som valfri, för gamla `.xls`-filer).

Vilka paket behöver vilket skript?

Paket	Behövs av
<code>openpyxl</code>	Alla rensnings-/sammanställnings-/CSV-skript, <code>payment_register.py</code> , samt Visma-avstämning mot Excel-fil
<code>pandas</code>	<code>visma_konvertera_betalningar.py</code> , <code>visma_register_inbetalningar_fixed.py</code> , <code>visma_import.py</code>
<code>pyautogui</code> , <code>pywinauto</code> , <code>pypclip</code>	<code>visma_register_inbetalningar_fixed.py</code> (skarp körning). <code>visma_import.py</code> använder <code>pyautogui</code> .
<code>playwright</code> (+ chromium)	Endast <code>payment_register.py</code>

Konfiguration

- **Kalibrering (Visma-inbetalningar):** `visma_inbetalningar_coordinates.json` skapas automatiskt när du kör `--calibrate` (se steg 3.9). Krävs för skarp körning.
- **Session (webbflödet):** `auth_state.json` skapas vid första manuella inloggningen i `payment_register.py`. Behandla filen som känslig – den innehåller inloggningscookies.
- **Konstanter i toppen av skripten:** t.ex. `KUNDID_X_RATIO` / `BOKFDAG_Y` i `visma_import.py` och `DRY_RUN` / `AMOUNT_TOLERANCE` / `MAX_INVOICES_PER_RUN` i inbetalningsskriptet. Ändras vid behov direkt i filen.

Lösa trådar & föräldralösa referenser

Under genomgången hittades tre referenser i koden till saker som inte finns i repot. Ingen av dem påverkar den terminalbaserade körningen i den här guiden, men de dokumenteras här så att de inte förvirrar. Se även ändringsloggen.

a) "Dashboard" och `config.json` – åtgärdat

Fyra rotskript hade tidigare kommentarer om att de "körs via dashboarden" trots att ingen dashboard fanns i repot, och `convert_betalningar_to_csv.py` refererade en `config.json / expected_output` som inte fanns.

Åtgärdat: `dashboard.py` är nu tillagd (se genvägen i avsnitt 3) och orkestrerar hela kedjan via de befintliga skripten. Kommentarererna i de fyra skripten är uppdaterade att peka på `dashboard.py`, och den felaktiga `config.json`-referensen är borttagen. Verktuget använder ingen extern `config.json` – mappkonventionen (`<arbetsmapp>\edit`) är inbyggd.

b) `visma_import\visma_config.json` – oanvänd koordinatfil

Filen innehåller klickpositioner (`x_ratio`, `y_offset`, `ctrl_class`) för fälten *Kund ID*, *Bokf.dag* och *Fak.belopp* – exakt de tre fält som `visma_import.py` använder. **Fakta:** `visma_import.py` läser *inte*

filen. Skriptet har i stället samma värden hårdkodade som konstanter (`KUNDID_X_RATIO` =0.036, `BOKFDAG_X_RATIO` =0.302, `FAK_BELOPP_X_RATIO` =0.789 m.fl.), som ligger mycket nära värdena i JSON-filen. Den enda kod som läste `visma_config.json` var en numera arkiverad version. **Slutsats:** föräldralös fil – ett utbrutet konfigurationsförsök som aldrig kopplades in. *Rekommendation:* ta bort filen, eller koppla in den om du vill kunna ändra `visma_import.py` :s positioner utan att röra koden.

Förväxla inte med den aktiva kalibreringen

Filen som verkligen används är `visma_inbetalningar_coordinates.json` (av `visma_register_inbetalningar_fixed.py`, för fälten *Bet.dag/Fakt.nr*). Den skapas med `--calibrate` – se steg 3.9. `visma_config.json` är en annan, oanvänd fil för ett annat skript.

3. Huvudflödet steg för steg

Stegen nedan står i körordning. Bankgiro- och Swish-spåren (steg 3.2–3.7) är oberoende – kör det/de spår du har data för. Byt ut `<MÅNADSMAPP>` mot din faktiska mapp, t.ex. `"C:\Ekonomi\Fak 2026\April"` (citattecken behövs när sökvägen innehåller mellanslag).

Genväg: kör allt via `dashboard.py`

I stället för att skriva varje kommando för hand kan du köra `python dashboard.py`. Det öppnar en meny som anropar exakt samma skript i rätt ordning med rätt `--input` / `--output`. Du sätter en **arbetsmapp** (där råfilerna ligger) en gång; rensade/sammanställda filer och CSV hamnar automatiskt i `<arbetsmapp>\edit`. Menyval **A** / **B** kör hela Bankgiro- respektive Swish-kedjan i ett svep. Stegen nedan förklarar vad varje menyval gör och hur man kör det fristående.

Genomgående mönster

Alla batch-skript tar `--input` (mapp att läsa från) och `--output` (mapp att skriva till), rör aldrig originalfilerna, hoppar över temporära Excel-låsfilerna (`~$...`) och skriver en sammanställning i terminalen på slutet. Datumstämplade utdatafiler får ett löpnummer (`_2` , `_3` ...) om namnet redan finns.

1 Exportera rådata (Bankgiro & Swish)

Syfte Skaffa källfilerna som resten av flödet bygger på.

Bankgiro Ladda ner insättningsuppgifterna som Excel. Filnamnet måste matcha mönstret `Bg*_Insättningsuppgifter*.xlsx`.

Swish I SEB internetbank: filtrera på *Swish + Inkommande* och ladda ner som Excel. Filnamnet måste matcha `Transaktioner * Swish*.xlsx`. (Den tidigare PDF-guiden `filtrera_swish_inbetalningar.pdf` visar skärmbilder för detta.)

Utdata Lägg råfilerna i din månadsmap.

2

Rensa Bankgiro-filer BANKGIRO

Syfte Tar bort brusceller och kolumnen *Bankgironummer/Avinummer*, behåller datum, belopp, löpnummer och betalningstabellen.

Kommando

```
python clean_bankgiro_files.py --input "<MÅNADSMAPP>" --output "<MÅNADSMAPP>\edit"
```

Indata

Bg*_Insättningsuppgifter_*.xlsx

Utdata

clean_bet_lista_<originalnamn>.xlsx i utdatamappen.

Utan flaggor körs bara ett inbyggt test

Kör du `python clean_bankgiro_files.py` helt utan argument bearbetas bara en hårdkodad testfil (Bg819-5968_Insättningsuppgifter_20260415.xlsx) om den råkar finnas. För skarp körning måste du ange `--input / --output` (eller `--all`).

Vanliga fel

- "Inga filer matchade ..." → fel mapp eller fel filnamn. Kontrollera att filen börjar på Bg och innehåller Insättningsuppgifter.
- *WARNING: "Sektionen ... saknas"* → filen har oväntad layout; öppna resultatet och kontrollera manuellt.

3

Rensa Swish-filer SWISH

Syfte Behåller kolumnerna Bokförd, Typ, Avsändare, Meddelande, Insättningar, Summa och lägger perioden överst.

Kommando

```
python clean_swish_files.py --input "<MÅNADSMAPP>" --output "<MÅNADSMAPP>\edit"
```

Indata

Transaktioner * Swish*.xlsx

Utdata

clean_swish_lista_<periodslut>.xlsx (om ingen period hittas används dagens datum).

Utan `--output` skrivs till <input>\edit.

Vanliga fel

- "Kunde inte hitta tabellen ..." → rubrikraden saknar de förväntade kolumnerna. Kontrollera att exporten har rubrikerna Bokförd/Avsändare/Insättningar.

4

Sammanställ Bankgiro **BANKGIRO**

Syfte Slår ihop alla rensade Bankgiro-filer till en fil med Datum, Avsändare, Betalningsreferens, Belopp, Löpnummer och Total.

Kommando

```
python sammanstall_betalningar.py --input "<MÅNADSMAPP>\edit" --output "<MÅNADSMAPP>\edit"
```

Indata `clean_bet_lista_*.xlsx`

Utdata `samlade_betalningar_<dagens datum>.xlsx` (utan `--output` skrivs den i indata-mappen).

Vanliga fel

- *VARNING: "Löpnummer saknas" / "Datum ... kunde inte tolkas"* → posten kom med men kontrollera källfilen; raden kan behöva rättas innan CSV-steget.

5

Sammanställ Swish **SWISH**

Syfte Slår ihop alla rensade Swish-filer; totalraden per källfil hoppas över automatiskt.

Kommando

```
python samla_swish_betalningar.py --input "<MÅNADSMAPP>\edit" --output "<MÅNADSMAPP>\edit"
```

Indata `clean_swish_lista_*.xlsx`

Utdata `samla_swish_bet_lista_<dagens datum>.xlsx` (utan `--output` = samma mapp som `--input`).

6 Skapa CSV för registrering (Bankgiro) BANKGIRO

Syfte Skapar den semikolonseparerade CSV:en och extraherar ett **5-siffrigt fakturanummer** ur betalningsreferensen. Skriver även en loggfil per rad.

Kommando

```
python convert_betalningar_to_csv.py --input "<MÅNADSMAPP>\edit"
```

Indata Mapp → senast ändrade `samlade_betalningar_*.xlsx`, eller peka direkt på en `.xlsx` -fil.

Utdata `betalningar_lista_to_reg_<datum>.csv` + `..._log.txt` (samma mapp som indata om `--output` utelämnas). Kolumner: Datum;Avsändare;Betalningsreferens;Fakturanummer;Belopp .

Valfritt `--output "<mapp>"`, `--date 2026-04-30` (styr datum i filnamnet).

Kontrollera loggen

Loggen visar per rad om fakturanummer hittades (OK), saknades eller om flera 5-siffriga tal fanns (då väljs det sista, status `WARNING`). Rätta referenser med saknat fakturanummer innan registrering.

7 Skapa CSV för registrering (Swish) SWISH

Syfte Samma CSV-format som Bankgiro, men från den samlade Swish-filen. Fakturanummer extraheras ur *Meddelande*.

Kommando

```
python step_4_skapa_swish_csv.py --input "<MÅNADSMAPP>\edit"
```

Indata Mapp → senast ändrade `samla_swish_bet_lista_*.xlsx` (eller direkt `.xlsx`).

Utdata `swish_betalningar_for_registrering_<datum>.csv` + `..._log.txt` .

Valfritt `--output` , `--date` .

8 Konvertera CSV till Visma-format

Syfte Byter kolumnnamn och ordning så att filen matchar det Visma-registreringen vill ha. Läser **alla** `.csv` i en mapp (både Bankgiro- och Swish-CSV kan ligga där).

Kommando (interaktivt – skriptet frågar efter mapparna)

```
python visma_import\visma_konvertera_betalningar.py
```

Frågor "Sökväg till mappen med indata-filer" och "mappen där resultatet ska sparas".

Kolumnmappning

Original (CSV)	Visma-namn (utdata)
Datum	Betalningsdatum
Avsändare	Kundnamn
Betalningsreferens	KundID
Fakturanummer	Fakturanr
Belopp	Belopp (oförändrat)

Utdata `<namn>_visma.csv` per fil (kolumnordning
Betalningsdatum;Fakturanr;Belopp;Kundnamn;KundID) + `konverteringslogg_*.txt` .

Vanliga fel

- "HOPPAR ÖVER ... saknar kolumner" → CSV:n har inte alla fem originalkolumner. Kontrollera att du pekar på CSV:erna från steg 6/7, inte råfilerna.

9 Registrera inbetalningar i Visma Compact

Syfte Halvautomatisk registrering i **Kundreskontra** → **Inbetalningar**. Skriver Bet.dag och Fakt.nr, kontrollerar beloppet exakt på öresnivå i dialogen *Rätt belopp?*, hanterar *Differens*, och loggar varje faktura. **Den slutliga bokföringen (Bankgiro/Bokför) gör du alltid själv.**

a) Verifiera att skriptet fungerar (ingen Visma-kontakt)

```
python visma_import\visma_register_inbetalningar_fixed.py --self-test
```

Ska avslutas med `Alla self-tester gick igenom.`

b) Kalibrera fälten första gången

Behövs när Visma inte exponerar fältetiketterna, när fönstret ändrat storlek, eller vid meddelandet "*Hittade inte Bet.dag-fältet*". Öppna Visma på **Kundreskontra** → **Inbetalningar** först, kör sedan:

```
python visma_import\visma_register_inbetalningar_fixed.py --calibrate
```

Följ instruktionerna: peka ut **Bet.dag** och **Fakt.nr**. Sparas i `visma_inbetalningar_coordinates.json`. Behåll samma fönsterstorlek vid kalibrering och körning.

c) Torrkörning (läser & kontrollerar, skriver inget)

```
python visma_import\visma_register_inbetalningar_fixed.py "<MÅNADSMAPP>\edit\ ... betalningar ... _visma.c
```

Skriptet frågar: "*Från vilket fakturanr vill du börja?*" (Enter = första raden) och "*Hur många fakturor vill du registrera?*". Utan `--live` skrivs inget i Visma (status `DRY_RUN`).

d) Skarp körning

Förbered Visma: ta säkerhetskopia, öppna Inbetalningar, välj *Obetalda*, kontrollera konto **1930**, håll fönstret synligt, stäng andra dialoger.

```
python visma_import\visma_register_inbetalningar_fixed.py "<...>_visma.csv" --live
```

Bekräfta genom att skriva exakt `ja`. Välj startfaktura och antal (högst 50). Kör **alltid en enda faktura** som första skarpa test.

Rör inte mus eller tangentbord under registreringen

Skriptet behöver kontroll över fokus. Vill du avbryta: **Ctrl+C** i terminalen, eller aktivera `pyautogui-failsafe`. Skriptet fortsätter aldrig till nästa faktura efter ett kritiskt fel.

Indata *_visma.csv (kolumner Betalningsdatum, Fakturanr, Belopp krävs; Kundnamn, KundID valfria). Även .xlsx/.xls/.xlsm går.

Utdata visma_inbetalningar_logg_<ÅÅÅÅMMDD_TTMMSS>.csv bredvid indatafilen.

Efter körning Läs loggen, kontrollera Vismas lista *Till betalning* >>, jämför mot filen och slutför Bankgiro/Bokför manuellt.

Ctrl+A i Visma

I Visma Compact öppnar **Ctrl** + **A** Artikelregistret i stället för "markera allt". Skriptet undviker det och rensar fält med **End** → **Shift** + **Home**. Gör likadant om du rättar manuellt.

10 Sammanställ & stäm av loggar (månadsvis)

Syfte Slår ihop alla körloggar för en månad/typ till en fil, markerar dubletter, kan skapa en dubblettrensad fil och stämma av mot en export över betalda fakturor. Kräver inga externa paket (openpyxl bara om jämförelsefilen är Excel).

Kommando (interaktivt)

```
python visma_import\visma_sammanstall_loggar.py
```

Frågor Mapp med loggfiler, månad/år (t.ex. 2026-06), betalningstyp (Bank eller Skatteverket). Därefter Ja/Nej för dubblettrensning respektive avstämning.

Indata visma_inbetalningar_logg_*.csv (endast poster med status OK och rätt månad tas med).

Utdata visma_inbetalningar_<typ>_<månad>_<år>_logg.csv , valfritt ..._clean_logg.csv och ..._avvikelser.csv .

Betalningstypen styr bara filnamnet

Det finns ingen kolumn som skiljer Bank från Skatteverket – urvalet sker på datum + status. Peka därför ut den mapp som innehåller rätt typ av loggar.

4. Alternativa & fristående flöden

4.1 Registrera i webbfakturasystemet i stället för Visma

payment_register.py är ett **alternativt slutsteg** till Visma-registreringen: det registrerar betalningar i det webbaserade fakturasystemet via Playwright, med en säkerhetskontroll som aldrig registrerar om Excel-beloppet inte matchar Att betala: på fakturasidan.

```
python payment_register.py
```

Skriptet frågar efter mappen med `total_betalningar_*.xlsx`, körläge (dry-run eller riktig – bekräftas med `REGISTRERA`) och antal rader (`1` eller `ALLA`). Första gången öppnas Chromium för manuell inloggning; sessionen sparas i `auth_state.json`. Utdata: `payment_log_*.csv` och ev. `screenshots/`.

Var kommer `total_betalningar_*.xlsx` ifrån?

Fakta: filnamnet refereras bara av `payment_register.py` (och `README.md`). **Inget skript i repot skapar en `total_betalningar_*.xlsx`** – rensningskedjan producerar `clean_bet_lista_*`, `samlade_betalningar_*`, `betalningar_lista_to_reg_*.csv` osv., men aldrig den filen. Webbflödet är alltså *inte* kopplat till den automatiska förkedjan: det konsumerar en separat, manuellt framtagna eller externt exporterad Excel-fil.

Filen måste innehålla kolumnerna som skriptet kräver: `Datum`, `Avsändare`, `Betalningsreferens` och `Belopp` (fakturanumret tolkas ur betalningsreferensen). Säkerhetskontrollen jämför `Belopp` mot `Att betala`: på fakturasidan innan något registreras.

Slutsats

Eftersom Visma är valt huvudmål beskrivs webbflödet som ett fristående alternativ. Vill du mata webbflödet från samma data som Visma-kedjan behöver ett litet extra steg skapa `total_betalningar_*.xlsx` ur de sammanställda filerna – det finns inte idag.

4.2 Registrera kundfakturer i Visma (Kundbokning)

`visma_import.py` är ett **fristående** verktyg – det registrerar kundfakturer (inte inbetalningar) helautomatiskt i Visma Kundbokning och tillhör inte inbetalningskedjan. Indata är en Excel-fil med kolumnerna `KundID`, `Fakturanr`, `Fakturadatum`, `Belopp`.

```
python visma_import\visma_import.py
```

Interaktivt: sökväg till Excel, antal fakturer (1–50) och startfakturanummer. Flöde per rad: `KundID` → `Bokf.dag` → `Fak.belopp` → `5×Tab` → `Num+`. Klickpositioner styrs av konstanterna `KUNDID_*` / `BOKFDAG_*` / `FAK_BELOPP_*` högst upp i filen. Utdata: `visma_import_logg_*.xlsx`.

5. Referens – alla skript, parametrar och flaggor

Rotmappen C:\ekonomi\scripts

Skript	Anrop & flaggor	In → Ut
dashboard.py	Interaktiv meny (inga flaggor). Sätter en arbetsmapp och anropar övriga skript i rätt ordning. A / B kör hela Bankgiro-/Swish-kedjan.	Orkestrerar de andra skripten – skapar inga egna filer
clean_bankgiro_files.py	--all (alla filer), --input <mapp> , --output <mapp> . Utan argument: bara inbyggd testfil. --input / --output / --all = batch.	Bg*_Insättningsuppgifter*.xlsx → clean_bet_lista*.xlsx
clean_swish_files.py	--input <mapp> , --output <mapp> (standard <input>\edit). Bearbetar alltid alla träffar.	Transaktioner * Swish*.xlsx → clean_swish_lista_<periodslut>.xlsx
sammanställ_betalningar.py	--input <mapp> , --output <mapp> (standard = input).	clean_bet_lista*.xlsx → samlade_betalningar_<datum>.xlsx
samlar_swish_betalningar.py	--input <mapp> , --output <mapp>	clean_swish_lista*.xlsx → samlar_swish_bet_lista_<datum>.xlsx

	(standard = input).	
<code>convert_betalningar_to_csv.py</code>	<pre>--input <mapp .xlsx> , --output <mapp> , --date ÅÅÅÅ-MM-DD , samt positionell filväg.</pre>	<pre>samlade_betalningar_*.xlsx → betalningar_lista_to_reg_<datum>.csv + logg</pre>
<code>step_4_skapa_swish_csv.py</code>	<pre>--input <mapp .xlsx> , --output , -- date , positionell filväg.</pre>	<pre>samla_swish_bet_lista_*.xlsx → swish_betalningar_for_registrering_<datum>.csv + logg</pre>
<code>payment_register.py</code>	Interaktivt (ingen flagga): mapp, dry-run/riktig (REGISTRERA), antal (1 / ALLA).	<pre>total_betalningar_*.xlsx → payment_log_*.csv (+ screenshots/)</pre>

Mappen visma_import\

Skript	Anrop & flaggor	In → Ut
visma_konvertera_betalningar.py	Interaktivt: indata-mapp, utdata-mapp.	alla .csv → *_visma.csv + konverteringslogg
visma_register_inbetalningar_fixed.py	[inputfil] , --self-test , --calibrate , --inspect , --debug-gui , --live (skarpt, annars DRY_RUN), --rader N (0 = alla).	*.xlsx/.xls/.xlsm/.csv → visma_inbetalningar_logg_*.csv
visma_import.py	Interaktivt: Excel-väg, antal (1–50), startfakturanr. Positioner via konstanter i filen.	*.xlsx (KundID, Fakturanr, Fakturadatum, Belopp) → visma_import_logg_*.xlsx
visma_sammanstall_loggar.py	Interaktivt: mapp, månad/år, typ (Bank/Skatteverket), Ja/Nej för rensning & avstämning.	visma_inbetalningar_logg_*.csv → *_logg.csv , *_clean_logg.csv , *_avvikelser.csv

Statusvärden i loggarna

Källa	Status	Betydelse
Visma-inbetalningar	OK	Fakturan registrerades i arbetslistan.
	DRY_RUN	Simulerad – inget skrevs i Visma.
	FEL / STOPPAD / HOPPAD	Fel upptäcktes / körningen avbröts / raden hoppades över.
Webbregistrering (payment_register.py)	DRY_RUN_OK / REGISTRERAD	Simulerad OK / registrerad på riktigt.
	BELOPP_MATCHAR_INTE	Excel-belopp ≠ Att betala: .
	REDAN_BETALD / SAKNAR_FAKTURANUMMER	Fakturan redan betald / inget fakturanr kunde tolkas.
	TIMEOUT / FEL / OKLAR_STATUS	Tidsgräns / fel / gick ej att verifiera.

6. Felsökning

Symptom / meddelande	Orsak	Åtgärd
<code>ModuleNotFoundError: pandas / pyautogui ...</code>	Beroende saknas (<code>requirements.txt</code> är ofullständig).	Kör installationskommandot i avsnitt 2.
"Inga filer matchade ..."	Fel mapp eller filnamn matchar inte mönstret.	Kontrollera <code>--input</code> och att råfilernas namn följer mönstret (t.ex. börjar på <code>Bg</code> / innehåller <code>Swish</code>).
Skriptet gör "inget" utan argument	<code>clean_bankgiro_files.py</code> kör bara testfilen utan flaggor.	Ange <code>--input</code> / <code>--output</code> eller <code>--all</code> .
"HOPPAR ÖVER ... saknar kolumner" (konvertering)	CSV:n har inte alla fem originalkolumner.	Peka på CSV:erna från steg 6/7, inte råfilerna.
<code>Hittade inte Bet.dag-fältet</code>	Visma exponerar inte fältet tekniskt.	Kör <code>--calibrate</code> och peka ut fälten igen.
Klick hamnar i fel fält	Visma-fönstrets storlek/layout ändrad.	Kalibrera om med samma fönsterstorlek som ska användas.
Visma-fönstret hittas inte	Visma stängt eller fel vy.	Öppna Visma Compact 6 → Kundreskontra → Inbetalningar (resp. Kundbokning för <code>visma_import.py</code>).
<code>Rätt belopp?</code> öppnas inte	Fakturanr saknas bland obetalda eller fel vy.	Kontrollera fakturanumret och välj <i>Obetalda</i> .
Beloppet kan inte verifieras (t.ex. <code>1465,01</code>)	Visma visar ett annat värde.	Bekräfta inte manuellt. Kontrollera beloppet och kör en enda faktura igen.
Status <code>DRY_RUN</code> trots att du vill skriva	<code>--live</code> saknas.	Lägg till <code>--live</code> – först efter godkänd torrkörning.
Dubblerat fakturanummer stoppar körningen	Samma fakturanr flera gånger i urvalet.	Rätta indatafilen innan körning.
Webbflödet: <code>BELOPP_MATCHAR_INTE</code>	Excel-belopp ≠ Att betala: .	Kontrollera beloppet i källfilen; raden registreras avsiktligt inte.
Svenska tecken (å ä ö) blir fel i Excel	Fel teckenkodning vid öppning.	Filerna skrivs som UTF-8 med BOM (<code>utf-8-sig</code>) och semikolon; öppna via Excels dataimport om det ändå blir fel.

7. Ordlista

Term	Betydelse
Rensad fil (<code>clean_*</code>)	Råfil där brus tagits bort och bara relevanta kolumner behållits.
Sammanställd fil (<code>samlade_*</code> , <code>samla_swish_*</code>)	Flera rensade filer hopslagna till en, med total per källfil.
Gemensamt CSV-format	<code>Datum;Avsändare;Betalningsreferens;Fakturanummer;Belopp</code> – bryggan mellan förkedjan och registreringen.
Visma-format	<code>Betalningsdatum;Fakturanr;Belopp;Kundnamn;KundID</code> – det <code>visma_register_inbetalningar_fixed.py</code> läser.
Torrkörning / DRY_RUN	Skriptet läser och kontrollerar men skriver inget i Visma. Standardläge utan <code>--live</code> .
Skarp körning (<code>--live</code>)	Skriptet skriver på riktigt i Visma. Kräver bekräftelse (<code>ja</code>).
Kalibrering	Sparade klickpositioner för Bet.dag/Fakt.nr (<code>visma_inbetalningar_coordinates.json</code>).
Differens / Restbelopp	Visma-dialog när betalt belopp $\neq$ fakturabelopp; skillnaden läggs som restbelopp på fakturan.
Avstämning	Jämförelse av Visma-loggen mot en export över betalda fakturor (<code>*_avvikelser.csv</code>).
Inbetalning vs faktura	Inbetalning = kund betalar en faktura (Kundreskontra→Inbetalningar). Faktura = ny kundfaktura (Kundbokning, <code>visma_import.py</code>).

8. Ändringslogg & beslut som behövs

Vad som slogs ihop

- Den fristående **tutorialen** (`docs/visma_import_tutorial.html/pdf` , om `visma_import.py`) och översiktssidan (`visma_import/docs/index.html`) ersätts av detta dokument.
- Innehåll från `Manual_Visma_inbetalningar.md` , `visma_sammanstall_loggar.md` , `visma_konvertera_betalningar.html` och `README.md` är sammanfört och lagt i körordning.
- Den tidigare odokumenterade Bankgiro/Swish-förkedjan (7 rotskript) är nu dokumenterad i huvudflödet.

Vad som ströks / rättades mot koden

- `requirements.txt` är kompletterad med `pandas` , `pyautogui` , `pywinauto` och `pyperclip` (tidigare saknades de trots att Visma-skripten kräver dem).
- Referensen till `visma_register_inbetalningar_old.py` som "aktivt skript" (i `index.html`) är struken – filen ligger i arkiv.
- Frågeordningen är beskriven per skript: `visma_import.py` frågar antal → startnr, medan inbetalningsskriptet frågar startnr → antal.

Behöver ditt beslut (OKLART)

#	Fråga	Status
1	Mappen <code>visma_import\archiv\</code> behandlas som arkiv och är utelämnad.	Bekräftat
2	<code>dashboard.py</code> tillagd (CLI-meny som orkestrerar hela kedjan). Kommentarererna i de fyra rotskripten pekar nu på den, och den döda <code>config.json</code> -referensen är borttagen. Se Lösa trådar (a) och genvägen i avsnitt 3.	Åtgärdat
3	<code>visma_config.json</code> är oanvänd – <code>visma_import.py</code> har positionerna hårdkodade. Dokumenterat i Lösa trådar (b). Kvarstår: ta bort filen eller koppla in den.	Dokumenterat
4	<code>total_betalningar_*.xlsx</code> skapas inte av något skript – extern/manuell fil för webbfliödet. Dokumenterat i avsnitt 4.1. Kvarstår: bygg ev. ett steg som skapar filen om webbfliödet ska matas från kedjan.	Dokumenterat
5	<code>requirements.txt</code> kompletterad med <code>pandas</code> , <code>pyautogui</code> , <code>pywinauto</code> , <code>pyperclip</code> (+ valfri <code>xlrd</code>).	Åtgärdat

Arbetsflödesguide för `C:\ekonomi\scripts` . Sanningskälla är koden; vid avvikelse följer guiden koden och noterar det. Ersätter tidigare tutorial och översiktssida.